

Dunes Insolites

Local dev setup — simple, verified, fully automatic catalog seeding

This is the final, simplified setup as of 2026-09-21 (branch `local-env`). Every claim below was verified by actually wiping the local database and Docker volumes completely and re-running the exact commands from nothing — twice — not assumed from reading code. Once the one-time config files exist (Section 2), day-to-day work is exactly four commands, and the database — including the real product catalog — builds itself.

The whole day-to-day workflow:

```
cd backend && docker compose up -d # postgres, keycloak, rabbitmq – nothing else
npm run backend:run # or just click Run in IntelliJ
npm run dev # frontend, localhost:3000
npm run dev:admin # backoffice, localhost:3100
```

No seed scripts to remember, no environment variables to export, no manual database restore. The real product catalog (2 nuitées, 6 extras, 3 accommodation tiers) and a set of safe sample records appear automatically the first time the backend starts against an empty database.

1. What's in this project

Part	Tech	Runs on	Purpose
frontend/	Next.js 16	localhost:3000	Public vitrine + espace client
admin/	Next.js 16	localhost:3100	Backoffice
backend/	Spring Boot 4 / Java 21	localhost:8080	API + business rules

Postgres/Keycloak/RabbitMQ run in Docker. The backend runs directly (Maven or an IDE), not in Docker — `docker-compose.yml` also defines a backend container and 3 Angular apps used only on the real staging/VPS hosts; a committed override file (Section 3) keeps those out of local dev automatically.

2. Prerequisites

- **Node.js 20+** and npm.
- **Java 21 (JDK)**.
- **Docker Desktop** — must be running before Step 1 below. It does not auto-start with Windows on this machine.

2. One-time setup (per machine)

These files are gitignored on purpose (they hold local passwords) and are **not** included when you pull the branch — create them once per machine. If you're on the same machine these were already created on, skip straight to Section 4.

backend/.env

```
POSTGRES_PASSWORD=localdev123
RABBITMQ_PASSWORD=localdev123
KEYCLOAK_ADMIN_USERNAME=admin
KEYCLOAK_ADMIN_PASSWORD=localdev123
KC_HOSTNAME=localhost
SPRING_MAIL_USERNAME=
SPRING_MAIL_PASSWORD=
KEYCLOAK_CLIENT_SECRET=*****
SEED_ADMIN_EMAIL=admin@dunes.local
SEED_ADMIN_PASSWORD=LocalAdmin123!
SEED_CAMPING_EMAIL=camping@dunes.local
SEED_CAMPING_PASSWORD=LocalCamping123!
APP_FRONTEND_URL=http://localhost:3000
```

The client secret is literally ten asterisks — the committed Keycloak realm export has its real secret scrubbed to that exact string before being committed, and Keycloak imports it verbatim, so the running realm's actual secret *is* that string.

backend/src/main/resources/application-local.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5434/duneinsolite
    password: localdev123
  rabbitmq:
    host: localhost
    password: localdev123
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8180/realms/duneinsolite
  mail:
    username: ""
    password: ""

keycloak:
  admin:
    server-url: http://localhost:8180
    client-secret: "*****"

seed:
  admin:
    email: admin@dunes.local
```

```
password: LocalAdmin123!
camping:
  email: camping@dunes.local
  password: LocalCamping123!

app:
  environment: local
  frontend:
    url: http://localhost:3000
```

Loaded automatically by a committed line in `application.yml` (`spring.config.import: optional:classpath:application-${DEV_ENV:local}.yml`) — no active Spring profile, no IDE run-config, no exported shell variable needed. **The** `app.environment: local` **line matters beyond just labeling** — it's the switch that gates the automatic database seeding in Section 4; without it, the real catalog / sample data never auto-restores.

frontend/.env.local

```
NEXT_PUBLIC_API_URL=http://localhost:8080/api
```

The /api suffix is required, not optional. `lib/api.ts`'s own paths (e.g. `/public/activities`) don't repeat it, so the base URL has to carry it. Confirmed live: leaving it off doesn't 404 — Spring Security's catch-all (`anyRequest().authenticated()`) returns a generic 401 for the unprefixed path instead, which `authProxy.ts` then reports as "Invalid email or password." — a real, misleading symptom for what is actually a URL config bug, not a credentials problem.

admin/.env.local

```
KEYCLOAK_ISSUER_URL=http://localhost:8180/realms/duneinsolite
KEYCLOAK_CLIENT_ID=duneinsolite-api
KEYCLOAK_CLIENT_SECRET=*****
ADMIN_BASE_URL=http://localhost:3100
NEXT_PUBLIC_API_URL=http://localhost:8080/api
NEXT_PUBLIC_FRONTEND_URL=http://localhost:3000
```

Same `/api` requirement applies here — `admin/lib/api.ts` uses the identical convention.

Templates for all three (`.env.local.example` / `.env.example`) are committed alongside each app, documenting exactly which variables each one needs.

3. Exact commands

Step 1 — Start Docker Desktop, then the containers

```
cd backend
docker compose up -d
```

This now works exactly as written — a committed `docker-compose.override.yml` (Docker Compose's own built-in auto-merge convention, applied only to a bare local up with no `-f` flags) keeps the unrelated staging/VPS-only services (`backend`, `partner-app`, `admin-app`, `camping-app` — which build from a sibling folder that doesn't exist in this checkout) out of a plain local run. Staging and the VPS are unaffected — they always pass explicit `-f` flags, which bypasses this file entirely.

Postgres needs ~45-60 seconds after first boot before its healthcheck passes. If Keycloak reports dependency `postgres failed to start` on the very first run against a brand new volume, just re-run the same command once `docker ps` shows Postgres as healthy.

Step 2 — Install JS dependencies (first time only, from repo root)

```
npm install
```

Step 3 — Run the backend

```
node scripts/mvn.mjs -q spring-boot:run
```

Or just click Run on IntelliJ's DuneinsoliteApplication Spring Boot configuration — no profile, no environment variables, nothing to configure. Verified by actually running it both ways.

The first time this runs against an empty database, it automatically restores the real product catalog and sample data — see Section 4. Every time after, that step is a no-op (guarded, like every other seed step) and startup just runs Flyway + the app.

Step 4 — Run the frontend and admin

```
npm run dev # localhost:3000
npm run dev:admin # localhost:3100
```

4. What builds itself automatically, and why it's safe

On the very first backend startup against an empty local database, `Seed.java` restores `backend/src/main/resources/seed-data/local-dev-dump.sql` — a checked-in, data-only SQL snapshot — before running its own existing seed steps. Gated twice:

- `app.environment == "local"` (set only by `application-local.yml`, Section 2) — this can never run against staging or production, so none of this dev/placeholder data can ever land in a real deployment.
- `tourTypeRepository.count() < 2` — the idempotency guard. The dump has fixed UUIDs; restoring it twice would violate primary-key constraints, so this only ever fires once, against a genuinely empty database.

What's actually in the dump, and where it came from:

Data	Source
2 tour types (nuitées), 6 extras, 3 accommodation tiers	REAL published content — created by running the repo's own <code>seed-tourtypes.mjs</code> / <code>seed-extras.mjs</code> / <code>seed-accommodations.mjs</code> , which pull straight from <code>frontend/lib/data/*-i18n/*.ts</code>
3 tours, 5 sources, 5 gallery photos, 6 nav items	Seed.java's pre-existing catalog data
3 guide profiles, 3 driver profiles, 3 content blocks, 3 newsletter subscribers	New Seed.java sample data — clearly fake (dunes.local emails), never shown on the public vitrine

Deliberately never in the dump, and why:

- **users / driver_profiles** — Keycloak-linked. A static dump's UUIDs wouldn't match whatever Keycloak instance restores it into (auth would silently break). `Seed.java` recreates these correctly through the real account-creation path instead, every time.
- **reservations / invoices / transactions** — go through `ReservationServiceImpl`'s validation and state machine. A direct data dump would bypass that entirely and could produce logically-wrong data.
- **reviews** — never fabricate reviews (CLAUDE.md's own rule). Real reviews are entered by hand from what a real guest actually said.
- **role_permissions, spoken_languages, site_settings, pages, page_blocks** — already fully (re)created by Flyway baseline migrations on every fresh database (confirmed live: including these caused a real primary-key collision on first restore). Flyway already owns this data completely; the dump adds nothing by duplicating it.

Also restorable by hand at any time — e.g. to reset just this data without touching anything else, or after a `docker compose down -v`:

```
npm run backend:db:seed-dump
```

(`scripts/restore-dev-dump.mjs` — restores the same file into whichever Postgres container is running, via `docker exec ... psql`. Designed for an empty table set, same idempotency assumption as the automatic path — running it twice in a row will hit the same primary-key guards.)

5. How this was verified (not assumed)

Check	Result
Full volume wipe (docker compose down -v) + docker compose up -d, twice	started exactly postgres/keycloak/rabbitmq both times
Single backend startup against a genuinely empty DB, twice	real catalog + sample data auto-restored, zero manual steps
Row counts after auto-restore (13 tables checked)	every count matched expected exactly
npm run backend:test:unit	134/134 tests pass
npm run typecheck (frontend/admin/api-types)	clean
npm run validate:prod-config	46 checks passed
npm run scan:secrets	1073 files scanned, no credential literals
GET /api/tours, /api/extras, /api/tour-types	200, real data
Admin login redirect through Keycloak	reaches the real sign-in form, not an error page
POST http://localhost:3000/api/auth/login/ (client@dunes.local)	200, real session cookie set, role: CLIENT

6. Default logins

Every credential below was verified live — a real password-grant token request against Keycloak, not just "this is what Seed.java sets."

App accounts — via Keycloak (backoffice at localhost:3100, API, espace client)

Role	Email	Password
Admin	admin@dunes.local	LocalAdmin123!
Camping	camping@dunes.local	LocalCamping123!
Client	client@dunes.local	LocalClient123!
Chauffeur — Hedi Chaabane	hedi.chaabane@dunes.local	LocalChauffeur123!
Chauffeur — Karim Mejri	karim.mejri@dunes.local	LocalChauffeur123!
Chauffeur — Nizar Ayari	nizar.ayari@dunes.local	LocalChauffeur123!

Admin / Camping — reset by editing backend/.env and application-local.yml's seed: block before the very first backend startup (seeding is skipped on every startup after the accounts already exist).

Client — a plain UserRole.CLIENT account (Seed.java's seedClientAccount()), for exercising booking / "espace client" flows. Low-risk enough that, unlike Admin/Camping, it doesn't need the env-var indirection — hardcoded the same way the sample data below is.

Chauffeur — these are real, functioning driver accounts (driver_profiles, linked to a real Keycloak user with the CHAUFFEUR role), but the code path that creates them (KeycloakUserSyncService.adminCreateInvitedDriver) is a real production

security feature: it deliberately generates a random password and **never discloses it anywhere** — correct for a real invited driver, useless for local dev. `Seed.java`'s `seedDriverProfiles()` immediately overwrites it with a known value right after creation, local dev only.

Guide — no login exists

Unlike drivers, `GuideProfile` has no linked `User` account at all in this codebase — confirmed by reading the entity — it's a contact-info directory only (name/email/phone/languages), not something anyone can log in as. The 3 sample guide profiles below (Amira Trabelsi, Youssef Bel Haj, Sami Karray) exist as data but have no password because there is genuinely nothing to log into — this isn't a gap in this guide, it's how the feature is actually built. If a guide login becomes a real requirement later, that's a feature to build (giving `GuideProfile` a `User` relationship, same shape as `DriverProfile`), not a local-dev config fix.

Infrastructure consoles

Console	URL	Username	Password
Keycloak admin console (master realm)	localhost:8180	admin	localdev123
RabbitMQ management UI	localhost:15672	duneinsolite	localdev123
PostgreSQL (db: duneinsolite)	localhost:5434	postgres	localdev123

The Keycloak admin console login is under the master realm (switch realms top-left after logging in) — a different account from every app login above, which all live in the `duneinsolite` realm.

7. Everything running, at a glance

Service	URL	Started by
Public site	http://localhost:3000	npm run dev
Backoffice	http://localhost:3100	npm run dev:admin
API	http://localhost:8080	node scripts/mvn.mjs -q spring-boot:run, or IntelliJ
Keycloak admin console	http://localhost:8180	docker compose up -d
RabbitMQ management UI	http://localhost:15672	docker compose up -d
PostgreSQL	localhost:5434	docker compose up -d

8. Gotchas worth knowing (for whoever touches this next)

- **A missing `/api` suffix on `NEXT_PUBLIC_API_URL` reports as "Invalid email or password," not a config/404 error.** Confirmed live: both `frontend/` and `admin/` expect the base URL to already include `/api` (their own `lib/api.ts` paths never repeat it) — leave it off and Spring Security's catch-all returns a generic 401 for the now-unmatched path, which `authProxy.ts` faithfully translates into a misleading credentials error. If login fails with that message, check the URL before suspecting the password.
- **Regenerating the dump with PowerShell will silently corrupt it.** Confirmed live: `Get-Content/WriteAllLines` without explicit UTF-8 encoding turned Français into FranÃ§ais and mangled Arabic text — no error, no warning. Always use plain Unix tools (`sed/grep` in Bash) to touch this file.
- **`pg_dump`'s boilerplate preamble is dangerous for a JDBC-based restore.** It includes `SELECT pg_catalog.set_config('search_path', '', false);` — harmless in a throwaway `psql` session, but executed via a pooled JDBC connection (as `Seed.java`'s restore does) it empties the search path *for the lifetime of that pooled connection*, breaking every later unqualified query. Confirmed live: caused a genuinely confusing relation "users" does not exist error, seemingly unrelated to the dump restore that actually caused it. Strip everything from the top of a fresh `pg_dump` output through `SET row_security = off;` before using it here.
- **Don't add more tables to the dump without checking Flyway migrations first.** `grep -l "INSERT INTO" backend/src/main/resources/db/migration/*.sql` shows which tables Flyway already seeds on every fresh database (`site_settings`, `pages`, `page_blocks`, `spoken_languages`, and one row of extras, as of this writing) — dumping those too causes primary-key collisions on restore.
- `docker compose up -d` (no service names) previously failed trying to build 3 Angular apps from a sibling folder that doesn't exist in this checkout. Fixed via `backend/docker-compose.override.yml` — note this file is **intentionally committed**, overriding the older convention (still visible in git history) that files with this exact name are `personal/gitignored`. Don't re-gitignore it.
- Maven's `compile` goal does not clean orphaned files from `target/classes` — if you delete or replace a resource file (like the dump), run `node scripts/mvn.mjs -q clean compile`, not just `compile`, or a stale cached copy keeps getting used silently.

Generated 2026-09-21. Every command and file above was actually run against this checkout — including two full volume wipes and clean re-runs from nothing — not transcribed from documentation or assumed to work. Nothing described here has been committed or pushed on your behalf; review `git status` before doing either.